

Exercise 1

Parallel and Distributed Systems

Selected Topics of Python Programming

This exercise is not specifically about parallel and distributed systems. Instead, it aims to repeat the essentials of Python programming relevant for the remainder of this course and the next exercises.

Preliminaries: Make sure to have a working Python setup. This includes the essential toolchain of Python (incl. Python interpreter and package manager `pip`) and an IDE of your choice (preferably PyCharm or VSCode).

Task 1 – Preliminary Knowledge Assessment

Read through the following questions and think about answers. Confident in answering them all? Great! If not, no worries, we'll cover that in this exercise. Come back again after doing the programming tasks and re-evaluate whether you can now answer these questions.

1. Are you able to implement essential object-oriented concepts in Python (i.e., classes, inheritance, constructors)?
2. Is there a main method in Python?
3. What do you know about type safety in Python? What are type annotations?
4. How do you output a string that is composed of many variables and some static elements (e.g., commas)?
5. What is the purpose of the keywords `def`, `from`, `with`, and `global`?

Task 2 – Selected Topics of Python Programming

- a) In a file named `animal.py`, implement a class named `Animal` with the instance variables `name` (type `str`) and `age` (type `int`). Implement a constructor that initializes the `name` and `age` variables with values passed as parameters to the constructor. Also, implement an instance method named `speak` that outputs the string `I am <NAME> and I am <AGE> years old.` to the console.
- b) In a file named `giraffe.py`, implement a class named `Giraffe` that inherits from the class `Animal`. The constructor should take the parameters `name` (`str`), `age` (`int`), and `height` (`float`). The constructor should first call the constructor from the class `Animal` to initialize the attributes `name` and `age` and then initialize the instance variable `height`. Also, override the `speak` method in a way that first the output of the class `Animal` appears and afterwards the string `I am a giraffe, my height is <HEIGHT> meters..`
- c) In a file named `pig.py`, implement a class named `Pig` that inherits from the class `Animal`. The constructor should take the parameters `name` (`str`), `age` (`int`), and `weight` (`float`). The constructor should first check whether the values of the parameters `weight` and `age` are greater than 0. If not, raise an exception of type `ValueError`. If the values are valid, the constructor should call the constructor from the class `Animal` to initialize the attributes `name` and `age` and then initialize the instance variable `weight`. Also, override the `speak` method in a way that first the output of the class `Animal` appears and afterwards the string `I am a pig, my weight is <WEIGHT> kg..`

- d) In a file named `pigs.data`, copy the following lines:

```
PigA ,4 ,65.2
PigB ,6 ,123.1
PigC ,5 ,87.6
```

Each line represents a pig. The first column is the name, the second column is the age, and the third column is the weight of the pig.

The following tasks should all be solved in a file named `main.py`.

- e) Declare a global variable named `number_of_animals` and initialize with the value 0.
- f) Define a function named `createGiraffe` that takes the parameters `name` (type `str`), `age` (type `int`), and `height` (type `float`). It should create an object of type `Giraffe` by passing the parameters to the `Giraffe` constructor. Increase the global variable `number_of_animals` by 1 and return the created object. Use type annotations in the function header to clarify expected data types for parameters and return values.
- g) Define a similar function `createPig` for creating Pigs. Don't forget the type annotations.
- h) Define a function named `readPigsFromFile` that takes a parameter of type `str` representing a path to a file. This function should open the provided file path, read the file line by line, and create `Pig` objects from each line using the `createPig` function. The `Pig` objects should be collected in a list and this list should be returned by the function. Don't forget the type annotations in the method header.
- i) Define a function named `returnAnimalWithName` that takes two parameters, a list of `Animal` objects and a name (type `str`). Search through the provided list and return the first animal and has the queried name. If no such animal is in the list, return `None`. Use appropriate type annotations.
- j) Define a function named `getAnimalsOlderThan` that takes two parameters, a list of `Animal` objects and an age (type `str` or type `int`). If the age was provided as a string, convert it to `int` first. Search through the provided list and return a list of all animals than are older than the provided age. Return an empty list if no such animal exists. Use appropriate type annotations.
- k) Write a main method proofing that all of the previous implemented functions fulfill their intended task. Also validate that the global variable is updated correctly.